BÁO CÁO MÃ NGUỒN - LAB 02

Sinh viên thực hiện: Lê Viết Đức

MSSV: 23DTHA4 / 0533

Môn học: Thực hành Bảo mật thông tin nâng cao

Đường dẫn file: `lab-02\ex01\api.py`

```
from flask import Flask, request, jsonify
from cipher.caesar import CaesarCipher


app = Flask(__name__)
caesar_cipher = CaesarCipher()


@app.route("/", methods=["GET"])
def index():
    return "Caesar Cipher API is running"


@app.route("/api/caesar/encrypt", methods=["POST"])
def caesar_encrypt():
    data = request.json

    if data is None or "plain_text" not in data or "key" not in data:
        return jsonify({"error": "Missing plain_text or key"}), 400

    plain_text = data["plain_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    encrypted_text = caesar_cipher.encrypt_text(plain_text, key)
    return jsonify({"encrypted_message": encrypted_text})


@app.route("/api/caesar/decrypt", methods=["POST"])
def caesar_decrypt():
    data = request.json
```

```
    if data is None or "cipher_text" not in data or "key" not in data:
        return jsonify({"error": "Missing cipher_text or key"}), 400

    cipher_text = data["cipher_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    decrypted_text = caesar_cipher.decrypt_text(cipher_text, key)
    return jsonify({"decrypted_message": decrypted_text})


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Đường dẫn file: `lab-02\ex01\requirements.txt`

```
Flask>=2.3.2
```

Đường dẫn file: `lab-02\ex01\cipher\__init__.py`

```

```

Đường dẫn file: `lab-02\ex01\cipher\caesar\__init__.py`

```
from .alphabet import ALPHABET
from .caesar_cipher import CaesarCipher
```

Đường dẫn file: `lab-02\ex01\cipher\caesar\alphabet.py`

```
from string import ascii_uppercase

ALPHABET = list(ascii_uppercase)
```

Đường dẫn file: `lab-02\ex01\cipher\caesar\caesar_cipher.py`

```
from cipher.caesar import ALPHABET


class CaesarCipher:
    def __init__(self):
        self.alphabet = ALPHABET

    def encrypt_text(self, text: str, key: int) -> str:
        text = text.upper()
        alphabet_len = len(self.alphabet)
        encrypted_text = []

        for letter in text:
            if letter in self.alphabet:
                letter_index = self.alphabet.index(letter)
                output_index = (letter_index + key) % alphabet_len
                encrypted_text.append(self.alphabet[output_index])
            else:
                encrypted_text.append(letter)

        return "".join(encrypted_text)

    def decrypt_text(self, text: str, key: int) -> str:
        text = text.upper()
        alphabet_len = len(self.alphabet)
        decrypted_text = []

        for letter in text:
            if letter in self.alphabet:
                letter_index = self.alphabet.index(letter)
                output_index = (letter_index - key) % alphabet_len
                decrypted_text.append(self.alphabet[output_index])
            else:
                decrypted_text.append(letter)

        return "".join(decrypted_text)
```

Đường dẫn file: `lab-02\ex02\api.py`

```
from flask import Flask, request, jsonify
from cipher.vigenere import VigenereCipher


app = Flask(__name__)
vigenere_cipher = VigenereCipher()


@app.route("/", methods=["GET"])
def index():
    return "Vigenere API is running"


@app.route("/api/vigenere/encrypt", methods=["POST"])
def vigenere_encrypt_api():
    data = request.json

    if data is None or "plain_text" not in data or "key" not in data:
        return jsonify({"error": "Missing plain_text or key"}), 400

    plain_text = data["plain_text"]
    key = data["key"]

    if not isinstance(key, str) or key == "":
        return jsonify({"error": "Key must not be empty"}), 400

    if not key.isalpha():
        return jsonify({"error": "Key must contain only letters"}), 400

    encrypted_text = vigenere_cipher.vigenere_encrypt(plain_text, key)
    return jsonify({"encrypted_text": encrypted_text})


@app.route("/api/vigenere/decrypt", methods=["POST"])
def vigenere_decrypt_api():
    data = request.json

    if data is None or "cipher_text" not in data or "key" not in data:
        return jsonify({"error": "Missing cipher_text or key"}), 400

    cipher_text = data["cipher_text"]
    key = data["key"]

    if not isinstance(key, str) or key == "":
        return jsonify({"error": "Key must not be empty"}), 400

    if not key.isalpha():
        return jsonify({"error": "Key must contain only letters"}), 400
```

```
    decrypted_text = vigenere_cipher.vigenere_decrypt(cipher_text, key)
    return jsonify({"decrypted_text": decrypted_text})


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Đường dẫn file: `lab-02\ex02\requirements.txt`

```
Flask>=2.3.2
```

Đường dẫn file: `lab-02\ex02\cipher\__init__.py`

```

```

Đường dẫn file: `lab-02\ex02\cipher\vigenere\__init__.py`

```
from .vigenere_cipher import VigenereCipher
```

Đường dẫn file: `lab-02\ex02\cipher\vigenere\vigenere_cipher.py`

```
class VigenereCipher:
    def __init__(self):
        pass

    def vigenere_encrypt(self, plain_text, key):
        encrypted_text = ""
        key_index = 0

        for char in plain_text:
            if char.isalpha():
                key_shift = ord(key[key_index % len(key)].upper()) - ord("A")
```

```
                if char.isupper():
                    encrypted_text += chr((ord(char) - ord("A") + key_shift) % 26 + ord("A"))
                else:
                    encrypted_text += chr((ord(char) - ord("a") + key_shift) % 26 + ord("a"))

                key_index += 1
            else:
                encrypted_text += char

        return encrypted_text

    def vigenere_decrypt(self, encrypted_text, key):
        decrypted_text = ""
        key_index = 0

        for char in encrypted_text:
            if char.isalpha():
                key_shift = ord(key[key_index % len(key)].upper()) - ord("A")

                if char.isupper():
                    decrypted_text += chr((ord(char) - ord("A") - key_shift) % 26 + ord("A"))
                else:
                    decrypted_text += chr((ord(char) - ord("a") - key_shift) % 26 + ord("a"))

                key_index += 1
            else:
                decrypted_text += char

        return decrypted_text
```

Đường dẫn file: `lab-02\ex03\api.py`

```
from flask import Flask, request, jsonify
from cipher.railfence import RailFenceCipher


app = Flask(__name__)
railfence_cipher = RailFenceCipher()


@app.route("/", methods=["GET"])
def index():
    return "Rail Fence API is running"
```

```
@app.route("/api/railfence/encrypt", methods=["POST"])
def railfence_encrypt():
    data = request.json

    if data is None or "plain_text" not in data or "key" not in data:
        return jsonify({"error": "Missing plain_text or key"}), 400

    plain_text = data["plain_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    encrypted_text = railfence_cipher.rail_fence_encrypt(plain_text, key)
    return jsonify({"encrypted_text": encrypted_text})


@app.route("/api/railfence/decrypt", methods=["POST"])
def railfence_decrypt():
    data = request.json

    if data is None or "cipher_text" not in data or "key" not in data:
        return jsonify({"error": "Missing cipher_text or key"}), 400

    cipher_text = data["cipher_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    decrypted_text = railfence_cipher.rail_fence_decrypt(cipher_text, key)
    return jsonify({"decrypted_text": decrypted_text})


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Đường dẫn file: `lab-02\ex03\requirements.txt`

```
Flask>=2.3.2
```

Đường dẫn file: `lab-02\ex03\cipher\__init__.py`

```

```

Đường dẫn file: `lab-02\ex03\cipher\railfence\__init__.py`

```
from .railfence_cipher import RailFenceCipher
```

Đường dẫn file: `lab-02\ex03\cipher\railfence\railfence_cipher.py`

```
class RailFenceCipher:
    def __init__(self):
        pass

    def rail_fence_encrypt(self, plain_text, num_rails):
        if num_rails <= 1 or plain_text == "":
            return plain_text

        rails = [[] for _ in range(num_rails)]
        rail_index = 0
        direction = 1

        for char in plain_text:
            rails[rail_index].append(char)

            if rail_index == 0:
                direction = 1
            if rail_index == num_rails - 1:
                direction = -1

            rail_index += direction

        cipher_text = "".join("".join(rail) for rail in rails)
        return cipher_text

    def rail_fence_decrypt(self, cipher_text, num_rails):
        if num_rails <= 1 or cipher_text == "":
            return cipher_text

        rail_lengths = [0] * num_rails
        rail_index = 0
```

```
        direction = 1

        for _ in range(len(cipher_text)):
            rail_lengths[rail_index] += 1

            if rail_index == 0:
                direction = 1
            if rail_index == num_rails - 1:
                direction = -1

            rail_index += direction

        rails = []
        start = 0

        for length in rail_lengths:
            rails.append(list(cipher_text[start:start + length]))
            start += length

        plain_text = []
        rail_index = 0
        direction = 1

        for _ in range(len(cipher_text)):
            plain_text.append(rails[rail_index].pop(0))

            if rail_index == 0:
                direction = 1
            if rail_index == num_rails - 1:
                direction = -1

            rail_index += direction

        return "".join(plain_text)
```